Let’s experiment with transactions. We’ll start by creating a new database table. (Make sure your database supports transactions, or this code won’t work for you.)

rails80/e1/ar/transactions.rb

```
create_table :accounts, force: true do |t|
  t.string  :number
  t.decimal :balance, precision: 10, scale: 2, default: 0
end
```

Next, we’ll define a rudimentary bank account class. This class defines instance methods to deposit money to and withdraw money from the account. It also provides some basic validation—for this particular type of account, the balance can never be negative.

rails80/e1/ar/transactions.rb

```
class Account < ActiveRecord::Base
  validates :balance, numericality: {greater_than_or_equal_to: 0}
  def withdraw(amount)
    adjust_balance_and_save!(-amount)
  end
  def deposit(amount)
    adjust_balance_and_save!(amount)
  end
  private
  def adjust_balance_and_save!(amount)
    self.balance += amount
    save!
  end
end
```

Let’s look at the helper method, `adjust_balance_and_save!`. The first line simply updates the balance field. The method then calls `save!` to save the model data. (Remember that `save!` raises an exception if the object cannot be saved—we use the exception to signal to the transaction that something has gone wrong.)